

# Prometheus SoC

High-Level Synthesis of a RISC-V Controlled Softmax Accelerator on the PYNQ-Z1

Jesse Levine

Department of Electronics, Carleton University

March 28, 2026

# Motivation

- Softmax is small in source code but expensive on a simple integer-oriented processor.
- The software path must evaluate exponentials, accumulate a global sum, and normalize every output.
- In fixed-point software, that path is both slow and numerically fragile.
- The project goal was to replace that bottleneck with dedicated hardware while keeping a small RISC-V core as the system controller.

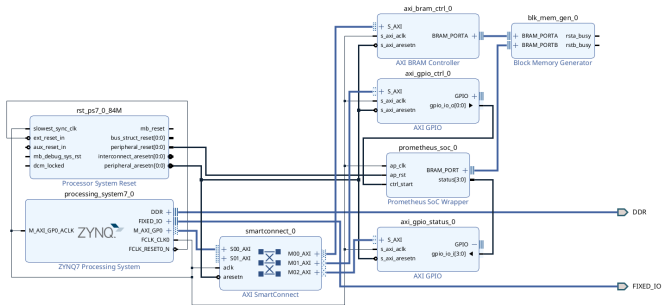
**Design question:** is the right endpoint a software-only core, a standalone accelerator, or a tightly integrated SoC?

# Project Evolution

- Report 1: reproduced the RV32I HLS baseline and the memory-image workflow.
- Report 2: added the multiplication support needed for practical fixed-point kernels.
- Report 3: defined the Softmax specialization strategy: shared memory, MMIO control, and approximation-based arithmetic.
- Report 5: produced the pipelined software baseline and a standalone accelerator.
- Report 6: integrated both into the final Prometheus SoC.

**Outcome:** the final result is a system-level architecture choice, not a kernel-only speedup.

## System Architecture

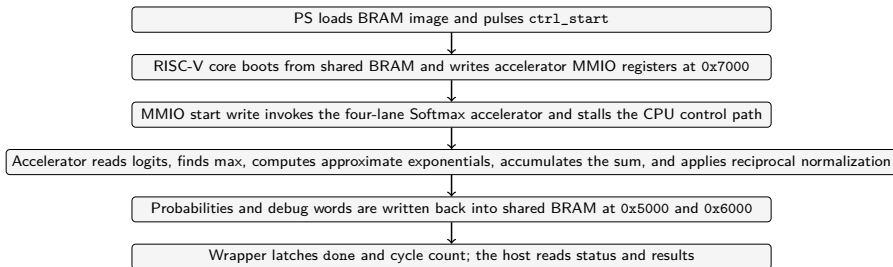


- Zynq PS7 hosts the system and reaches PL through M\_AXI\_GP0.
- AXI SmartConnect fans out to BRAM control, start GPIO, and status GPIO.
- Shared BRAM is dual-port and visible to both PS and the HLS SoC wrapper.
- The wrapper edge-detects start, latches done, and measures cycle count.

# Implemented Architecture Summary

| Item                | Implemented detail                                                                                                   |
|---------------------|----------------------------------------------------------------------------------------------------------------------|
| Top-level HLS block | One HLS SoC containing a 32-bit RISC-V controller core, a four-lane Softmax accelerator, and a shared BRAM image.    |
| Shared memory       | 16384 lines $\times$ 128 bits = 256 KiB true dual-port BRAM; each line holds four 32-bit words.                      |
| Board mapping       | PS7 accesses BRAM through AXI at 0x40000000; the HLS SoC uses the second BRAM port directly.                         |
| Local memory map    | Program at 0x0000; inputs at 0x4000; probabilities at 0x5000; debug at 0x6000; MMIO at 0x7000–0x7010.                |
| MMIO registers      | 0x7000 AP_CTRL; 0x7004 input base; 0x7008 probability base; 0x700C debug base; 0x7010 vector length $N$ .            |
| Status word         | Bits [3:0] give done, idle, ready, and busy; bits [31:4] hold the latched FPGA cycle count.                          |
| CPU subset          | RV32I integer core plus multiply instructions used by the software path; division and remainder are not implemented. |
| Launch semantics    | Writing the MMIO start bit causes the CPU control path to call the accelerator directly and block until completion.  |

# Runtime Control Flow



# Accelerator Datapath

- Signed Q16.16 fixed-point datapath
- Four-lane vector width
- Maximum supported vector length: 256
- Stage 1: stream logits and reduce max
- Stage 2: compute approximate exponentials after max subtraction
- Stage 3: cache exponentials and accumulate a 64-bit sum
- Stage 4: apply a 16-segment reciprocal approximation in Q2.30
- Stage 5: normalize and write probabilities back to BRAM

## Approximation strategy

$$e^x = 2^{x/\ln 2}$$

- 17-point piecewise-linear lookup for the fractional exponential term
- Power-of-two shift for the integer term
- HLS DATAFLOW to overlap exponentiation, buffering, normalization, and writeback

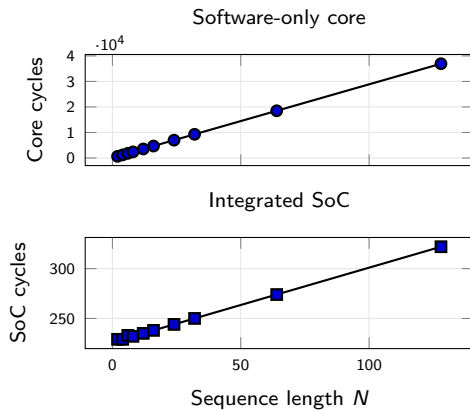
## Implementation-Level Resource Tradeoff

| Resource / Metric      | Core Only | SoC    |
|------------------------|-----------|--------|
| LUT                    | 2169      | 26090  |
| FF                     | 1465      | 52339  |
| DSP                    | 12        | 32     |
| BRAM                   | 0         | 9      |
| Post-route period (ns) | 11.345    | 11.514 |

- The SoC is much larger because it includes the accelerator datapath, local buffers, and the MMIO integration structure.
- The architectural question is whether this area cost is justified by throughput and numerical gains.
- The remainder of the results show that the answer is yes.

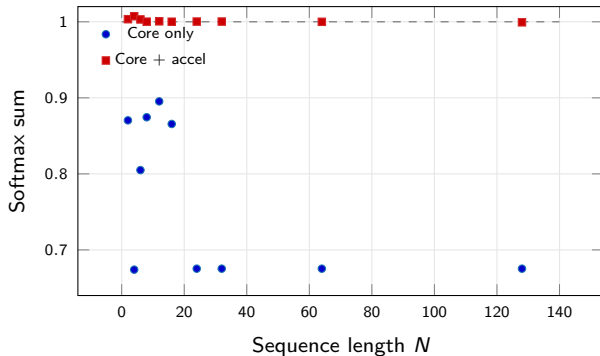


# Latency Scaling



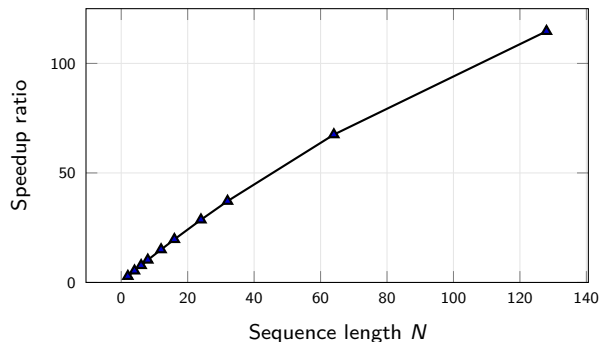
- The software-only baseline scales linearly with a very large slope.
- The integrated SoC keeps the per-element cost extremely small.
- Separate axes are necessary because a shared axis hides the SoC behavior.
- This throughput gap is what motivated the final architecture choice.

# Numerical Behavior



- The software-only path loses probability mass as  $N$  grows.
- Its Softmax sum collapses toward roughly 0.675 for longer vectors.
- The accelerator-based SoC keeps the sum close to 1.0.
- The SoC is therefore not only faster, but also materially better behaved numerically.

## Speedup Result



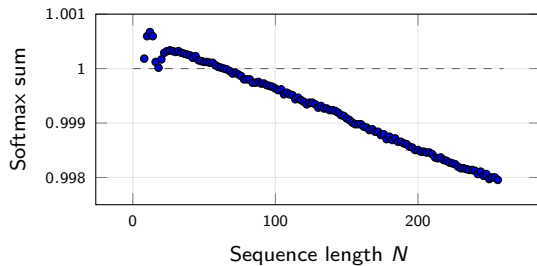
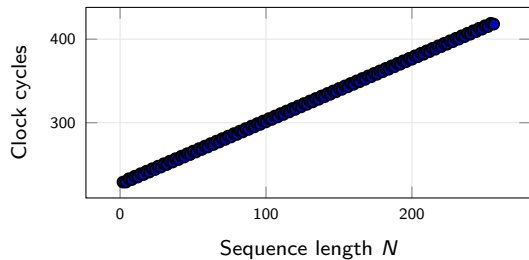
- Measured speedup at  $N = 128$ :  $114.7\times$
- Large- $N$  slope-based limit: **about**  $389\times$
- The SoC has a larger fixed setup cost.
- But its marginal cost per additional element is dramatically lower than the software baseline.

# FPGA Implementation

| Metric                | Implemented SoC  |
|-----------------------|------------------|
| Board                 | Digilent PYNQ-Z1 |
| FPGA                  | xc7z020c1g400-1  |
| Clock period (ns)     | 11.875           |
| Clock frequency (MHz) | 84.211           |
| WNS (ns)              | 0.332            |
| WHS (ns)              | 0.010            |
| Slice LUTs            | 32717            |
| Slice Registers       | 60358            |
| Block RAM Tiles       | 68.5             |
| DSPs                  | 32               |
| Timing status         | Met              |

- The complete board-integrated design closed timing on the target FPGA.
- This is not a simulation-only argument: the processor, BRAM, AXI glue, GPIO path, and accelerator were implemented and exercised together.
- Reported timing values come from the wrapper-level status-word measurement path.

# FPGA Validation



FPGA latency remains low across the full sweep, and the Softmax sum stays close to one. At representative points, FPGA cycle counts match simulation exactly.

## Takeaways

- Prometheus SoC is a tightly integrated RISC-V plus accelerator architecture built with HLS and validated on the PYNQ-Z1.
- The design uses a shared 128-bit BRAM image, MMIO-controlled launch semantics, and a wrapper that cleanly exposes done and cycle count to the host.
- The four-lane fixed-point accelerator removes the Softmax bottleneck while also improving numerical behavior relative to the software-only baseline.
- The final contribution is a complete, measured hardware-software co-design result rather than a standalone accelerator block.

# References

-  O. Toker, “A high-level synthesis approach for a RISC-V RV32I-based system on chip and its FPGA implementation,” *Engineering Proceedings*, 2023.
-  R. Kim, D. Lee, J. Kim, J. Park, and S. E. Lee, “Hardware accelerator for approximation-based softmax and layer normalization in transformers,” *Electronics*, 2025.
-  A. Vaswani et al., “Attention is all you need,” *NeurIPS*, 2017.